

Personal Expense System

Relazione Tecnica Integrata: Schema Logico, DDL, DML e Istruzioni Operative

Il presente documento costituisce la relazione tecnica finale e omnicomprensiva del database relazionale a supporto dell'applicazione **Personal Expense System**. All'interno sono integrati e strutturati in modo coerente tutti i moduli progettuali richiesti: la modellazione logica delle entità, gli script di definizione (DDL), i set di dati dimostrativi (DML) e la guida operativa per la corretta installazione ed esecuzione dell'applicativo.

Repository GitHub del Progetto:

Il codice sorgente completo, la cronologia dei commit e i file di configurazione dell'applicazione sono disponibili e liberamente consultabili al seguente indirizzo ufficiale:

<https://github.com/cc-1596/Project-Python.git>

Nota sulle correzioni sintattiche applicate:

Durante la fase di consolidamento dei sorgenti, sono stati eliminati alcuni refusi di sintassi SQL che avrebbero compromesso la stabilità del database locale SQLite:

- Nella tabella spese è stata rimossa la virgola eccedente posta subito dopo la dichiarazione dell'ultimo vincolo CHECK.
- Nella tabella budget è stata corretta la dichiarazione della chiave primaria composta PRIMARY KEY(data, categoria_id), eliminando il riferimento duplicato al campo categoria_id.

8.2. Schema Logico del Database

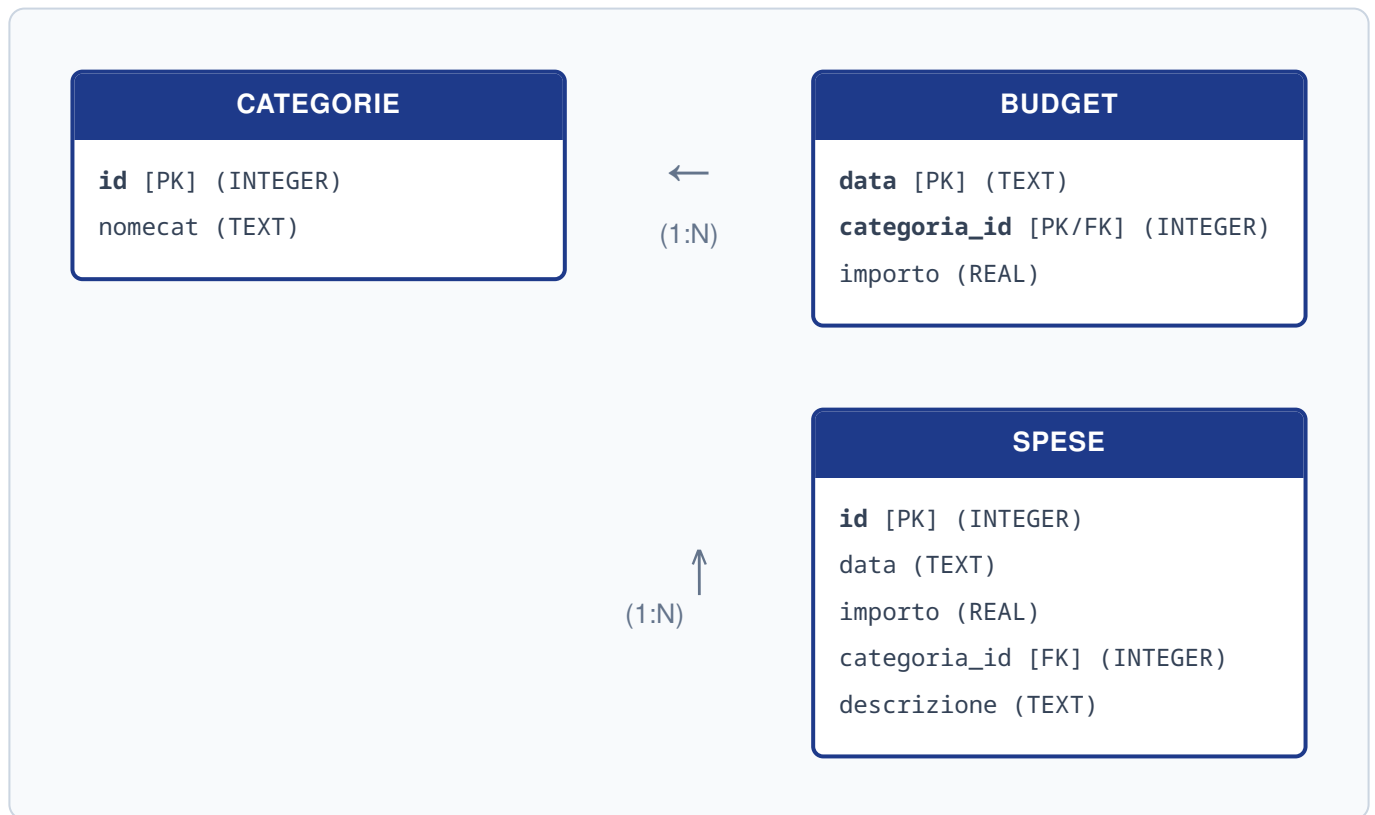
Lo schema logico descrive l'organizzazione strutturale dei dati e le relazioni di integrità referenziale che intercorrono tra le entità. Nella notazione lineare sottostante, le chiavi primarie (PK) sono sottolineate, mentre i campi contrassegnati in *corsivo* rappresentano le *chiavi esterne (FK)*.

Notazione Relazionale Lineare

- **CATEGORIE** (id, nomecat)
- **SPESE** (id, data, importo, categoria_id → CATEGORIE, descrizione)
- **BUDGET** (data, categoria_id → CATEGORIE, importo)

Rappresentazione Grafica dei Legami (Schema Relazionale)

Il seguente diagramma strutturale illustra i flussi relazionali e i punti di ancoraggio delle chiavi esterne dirette verso l'entità dizionario principale (CATEGORIE).



Descrizione Cardinalità e Integrità Referenziale

- **CATEGORIE (1) — (0,N) SPESE:** Relazione uno-a-molti. Una singola categoria di spesa può raccogliere nel tempo un numero indefinito (zero o più) di transazioni finanziarie reali. Ogni spesa inserita è tassativamente associata a una sola categoria esistente.
- **CATEGORIE (1) — (0,N) BUDGET:** Relazione uno-a-molti. Ogni categoria può essere pianificata all'interno del piano budget per più mensilità (es. un record di budget dedicato ad "Alimentari" per ogni mese del calendario). La chiave primaria accoppiata (data, categoria_id) impedisce la presenza di record duplicati per lo stesso mese.

Sezione A) Script di Creazione del Database (DDL)

Di seguito viene riportato lo script definitivo di generazione delle strutture fisiche e dei relativi vincoli di dominio su SQLite.

```
-- =====
-- SEZIONE A) SCRIPT DI CREAZIONE DEL DATABASE (DDL)
-- =====

-- Abilitazione del supporto alle chiavi esterne per SQLite
PRAGMA foreign_keys = ON;

-- 1. TABELLA: Categorie di spesa
CREATE TABLE IF NOT EXISTS categorie (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nomecat TEXT,
    -- Vincolo CHECK: impedisce l'inserimento di nomi vuoti o composti da soli spazi
    CONSTRAINT chk_nome_non_vuoto CHECK (length(trim(nomecat)) > 0)
);

-- 2. TABELLA: Spese effettive
CREATE TABLE IF NOT EXISTS spese (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    data TEXT NOT NULL,
    importo REAL NOT NULL,
    categoria_id INTEGER,
    descrizione TEXT,

    -- Vincolo FOREIGN KEY: integrità referenziale verso la tabella categorie
    FOREIGN KEY (categoria_id) REFERENCES categorie(id),

    -- Vincolo CHECK: l'importo deve essere strettamente maggiore di zero
    CONSTRAINT chk_importo_spesa_positivo CHECK (importo > 0),

    -- Vincolo CHECK: garantisce la consistenza formale della data (formato ISO 10 caratteri)
    CONSTRAINT chk_formato_data_spesa CHECK (data LIKE '____-__-__')
);

-- 3. TABELLA: Pianificazione Budget Mensile
CREATE TABLE IF NOT EXISTS budget (
    data TEXT NOT NULL,
    categoria_id INTEGER,
    importo REAL NOT NULL,

    PRIMARY KEY (data, categoria_id),

    -- Vincolo CHECK: l'importo del budget stanziato deve essere positivo
```

```

CONSTRAINT chk_importo_positivo CHECK (importo > 0),

-- Vincolo CHECK: garantisce la consistenza formale del mese (formato 7
caratteri)
CONSTRAINT chk_formato_data_budget CHECK (data LIKE '____-__'),

-- Vincolo FOREIGN KEY: garantisce l'integrità referenziale in cascata
CONSTRAINT fk_budget_categoria
    FOREIGN KEY (categoria_id) REFERENCES categorie(id)
    ON DELETE CASCADE ON UPDATE CASCADE,

-- Vincolo UNIQUE composto: previene record duplicati per mese/categoria
CONSTRAINT uq_data_categoria UNIQUE (data, categoria_id)
);

```

Sezione B) Script di Inserimento Dati di Esempio (DML)

Set di istruzioni DML per il popolamento di test coesistente con lo scenario di sfioramento budget analizzato.

```
-- =====
-- SEZIONE B) SCRIPT DI INSERIMENTO DATI DI ESEMPIO (DML)
-- =====

-- 1. POPOLAMENTO TABELLA CATEGORIE
INSERT INTO categorie (nomecat) VALUES ('Alimentari');
INSERT INTO categorie (nomecat) VALUES ('Trasporti');
INSERT INTO categorie (nomecat) VALUES ('Intrattenimento');

-- 2. POPOLAMENTO TABELLA BUDGET (Pianificazione Gennaio 2026)
INSERT INTO budget (data, categoria_id, importo) VALUES ('2026-01', 1, 300.00);
INSERT INTO budget (data, categoria_id, importo) VALUES ('2026-01', 2, 100.00);
INSERT INTO budget (data, categoria_id, importo) VALUES ('2026-01', 3, 150.00);

-- 3. POPOLAMENTO TABELLA SPESE
-- Scenario Categoria Alimentari (ID 1) -> Totale speso: 320.00€ (Superamento Budget di 20€)
INSERT INTO spese (data, importo, categoria_id, descrizione) VALUES ('2026-01-05',
150.50, 1, 'Spesa Esselunga');
INSERT INTO spese (data, importo, categoria_id, descrizione) VALUES ('2026-01-18',
169.50, 1, 'Supermercato Conad');

-- Scenario Categoria Trasporti (ID 2) -> Totale speso: 75.00€ (Sotto Soglia Budget)
INSERT INTO spese (data, importo, categoria_id, descrizione) VALUES ('2026-01-02',
50.00, 2, 'Ricarica Treno');
INSERT INTO spese (data, importo, categoria_id, descrizione) VALUES ('2026-01-15',
25.00, 2, 'Carburante');
```

Tabella di Sintesi dei Vincoli Architeturali

Tabella	Colonna / Nome Vincolo	Tipo di Vincolo	Descrizione Funzionale Interna
categorie / spese	id	PRIMARY KEY	Identificativo univoco auto-incrementale numerico di riga.
spese / budget	data, importo	NOT NULL	Campi mandatori bloccanti per impedire l'assenza di dati atomici vitali.
spese / budget	chk_importo_positivo...	CHECK	Regola di dominio che interdice valori monetari negativi o nulli (≤ 0).
spese / budget	chk_formato_data...	CHECK	Validazione formale a livello di stringhe mediante pattern fissi (LIKE).
spese / budget	categoria_id	FOREIGN KEY	Ancoraggio referenziale vincolato verso l'entità padre categorie.
budget	uq_data_categoria	UNIQUE	Garantisce l'esistenza di un solo stanziamento mensile per categoria.

9. Istruzioni Dettagliate per l'Esecuzione del Programma

9.1. Requisiti di Sistema

L'applicazione è interamente standalone, scritta in linguaggio procedurale/oggetti con **Python 3**. Sfruttando un motore database SQL incorporato (SQLite3), non necessita dell'attivazione di demoni o servizi server esterni in background.

- **Runtime:** Python SDK versione 3.10 o superiore (indispensabile per l'uso dei costrutti match-case).
- **Librerie Esterne:** Nessuna. L'applicativo si basa esclusivamente su moduli core nativi (sqlite3, os, datetime).

9.2. Topologia dei File di Progetto

Verificare la corretta disposizione dei sorgenti nell'alberatura delle cartelle locali prima dell'avvio, al fine di salvaguardare la risoluzione dei puntamenti relativi al DB:

```
PersonalExpenseSystem/  
├─ sql/  
│   └─ (In questa directory verrà inizializzato in automatico il file spese.db)  
└─ src/  
    ├─ main.py           (Entry-point ed orchestratore del menu)  
    ├─ categorie.py      (Modulo di logica - Gestione Categorie)  
    ├─ spese.py          (Modulo di logica - Inserimento Movimenti)  
    ├─ budget.py         (Modulo di logica - Allocazione Preventivi)  
    └─ report.py         (Modulo Analitico - Calcolo Scostamenti)
```

9.3. Istruzioni di Compilazione

Essendo Python un linguaggio prettamente **interpretato**, il codice sorgente non richiede alcuna fase transitoria di compilazione né la generazione di file intermedi in bytecode binario. I file vengono analizzati ed eseguiti direttamente a runtime.

9.4. Sequenza di Avvio e Comandi Esatti

Per mandare in esecuzione il programma, aprire il terminale di riga di comando (Prompt dei Comandi / PowerShell su Windows o Terminale di sistema su macOS/Linux) ed effettuare la seguente procedura in due passi:

1. Spostamento nella directory sorgente (src):

Utilizzare il comando di navigazione `cd` per posizionarsi all'interno della cartella dei codici sorgenti.

Esempio universale:

```
cd /percorso/locale/PersonalExpenseSystem/src
```

2. Esecuzione del comando esatto di avvio:

Invocare l'interprete Python sul file principale di gestione:

- Su sistemi operativi **Microsoft Windows**:

```
py main.py
```

- Su sistemi operativi **macOS / GNU Linux**:

```
python3 main.py
```

9.5. Automatismi al Primo Avvio

In fase di bootstrap, il file `main.py` rileva l'eventuale assenza della base dati: crea la cartella `./sql`, genera il file `spese.db`, ne sancisce l'attivazione dei vincoli referenziali (`PRAGMA foreign_keys = ON;`) e compila in sequenza le tre tabelle relazionali. Al termine, l'utente viene accolto dal menu interattivo testuale a cominciacia di cinque opzioni.